



Introduction to AI Systems

BITS Pilani

Dr. Minati Rath
mrath@wilp.bits-Pilani.ac.in



<SS ZG662, Introduction to AI Systems >

Session 4

These slides are prepared by the instructor, with grateful acknowledgement of many others who made their course materials freely available online.

AI Ecosystem and Model Marketplaces:

- Foundation Models,
- Open Models,
- APIs,
- Model Hubs &
- Build-vs-Buy



Foundation models are becoming infrastructure choices, not just software features.

Enterprise buyers now choose among proprietary APIs, managed open models, hosted inference and self-managed models.

The strategic question: how do we obtain model capability while preserving economics, governance and optionality?

Replace “Which model is best?” with “Which model wins for this workload?”

A production decision must combine capability, license, data boundary, latency, TCO, availability and exit options.

The model is only one component; inference provider and platform architecture can create equal or greater lock-in.

Application: copilots, search, agents, automation and decision support.

AI platform: gateway, RAG, evaluation, policy, observability and routing.

Model layer: GPT, Claude, Gemini, Llama, Mistral, Qwen, DeepSeek, Phi, Gemma and specialist models.

Compute layer: API infrastructure, managed endpoints, hosted GPUs and enterprise GPU clusters.



The enterprise AI market is no longer a single-vendor market.

Proprietary models: GPT, Claude, Gemini

Open/open-weight ecosystems: Llama, Mistral, Qwen, DeepSeek, Gemma, Phi

Model hubs: Hugging Face - broad model discovery, model cards, weights and community ecosystem.

Cloud model platforms: AWS Bedrock, Google Vertex AI, Microsoft Foundry, Amazon Bedrock - managed access to multiple model families; Bedrock Marketplace adds third-party/open model choices

Infrastructure ecosystems: NVIDIA NGC

Hosted inference providers: Together AI, Fireworks AI, Replicate and others

Enterprise question: Which capability should we buy, which should we build, and how do we retain the ability to switch?

The Enterprise AI Market Has Become a Multi-Layer Ecosystem



Layer	Examples	Enterprise decision
Foundation models	GPT, Claude, Gemini	Capability
Open models	Llama, Qwen, Mistral	Control / customization
Model hubs	Hugging Face	Discovery
Model marketplaces	Bedrock Marketplace	Procurement + deployment
Inference providers	Together, Fireworks	Serving economics
Cloud AI platforms	Bedrock, Vertex, Foundry	Integration
GPU ecosystem	NVIDIA NGC	Infrastructure
Applications	Copilots, agents, search	Business value

The New Enterprise Model-Selection Question



Instead of: “Which is the best AI model?”

Ask: For this workload:

Which model achieves the required quality?

Which deployment route meets data requirements?

Which provider meets latency and availability targets?

Which option produces the lowest cost per successful task?

Which license permits the intended business use?

How easily can we replace the model six months later?

Model selection is therefore a business architecture decision.

Major discovery and access routes

Hugging Face

Open/open-weight models, Model cards, Weights and files

Community ecosystem

AWS Bedrock

Managed access to multiple model providers

Amazon Bedrock Marketplace

Additional third-party and specialized models

AWS-managed deployment route

Google Vertex AI Model Garden

Google, partner and open models

Microsoft Foundry Model Catalog

Models integrated into Azure's AI platform

NVIDIA NGC

Models, containers and optimized AI software

AWS describes Bedrock Marketplace as a catalog containing 100+ popular, emerging and specialized foundation models.

Hugging Face: What Makes It Different?



An enterprise can find

Llama variants

- Qwen variants
- Mistral variants
- Gemma
- Phi
- DeepSeek
- Embedding models
- Rerankers
- Vision models
- Speech models
- Domain-specific models

The model page can expose

- Model card
- License
- Downloads
- Files
- Evaluation results
- Quantized versions
- Inference providers
- Community activity

The important point: finding a model is not the same as approving it for production.



Hugging Face is primarily an ecosystem and discovery layer.

An enterprise can find:

- Llama variants
- Qwen variants
- Mistral variants
- Gemma
- Phi
- DeepSeek
- Embedding models
- Rerankers
- Vision models
- Speech models
- Domain-specific models

The model page can expose:

- Model card
- License
- Downloads
- Files
- Evaluation results
- Quantized versions
- Inference providers
- Community activity

The important point: finding a model is not the same as approving it for production.



- Creator | Who is responsible for the model?
- Base model | What is this model derived from?
- Version | Which exact checkpoint are we evaluating?
- License | Can we commercially use it?
- Model card | What limitations are disclosed?
- Evaluation | What evidence supports capability claims?
- Files | Can we actually deploy it?
- Quantization | What hardware requirements apply?
- Inference provider | Who can serve it for us?
- Downloads | Is there ecosystem adoption?

Important: downloads and likes are signals of interest—not evidence of enterprise readiness.

Model Card as a Procurement Artifact



A model card should become part of the AI procurement record.

Capture: Identity

- Organization
- Model family
- Version
- Parameter size
- Architecture

Usage

- Intended use
- Known limitations
- Supported languages
- Supported modalities

Technical

- Context length
- Quantization options
- Hardware requirements
- Inference framework

Commercial

- License
- Usage restrictions
- Redistribution terms

Evidence

- Benchmarks
- Evaluation methodology
- Known weaknesses

A model can have: High downloads

but: • poor documentation

- restrictive license
- weak evaluation
- poor inference efficiency
- unsupported architecture
- no enterprise SLA
- unclear provenance

Therefore:

Discovery → Inspection → Evaluation → Security → Legal → Pilot → Production

This is why a model hub should be treated as a research/procurement discovery environment, not automatically as an approved production catalog.



Bedrock changes the architecture from: Application → Model Provider to: *Application → Bedrock → Multiple Models*

Potential benefits:

- Common API layer
- Managed inference
- Cloud IAM integration
- Governance
- Multiple model families
- AWS-native tooling
- Easier experimentation

The strategic benefit is reducing the amount of application code tied directly to one model provider.

What Models Can an Enterprise Encounter in Bedrock?



Examples include:

Anthropic

- Claude family

Meta

- Llama family

Amazon

- Nova

- Titan

Mistral

- Mistral models

Cohere

- Command models

AI21

- Jamba / other models

DeepSeek

- DeepSeek variants

Bedrock's current model documentation shows a broad catalog with provider, API, endpoint and regional availability information.

Bedrock Marketplace extends the model catalog.

Enterprise workflow:

Discover → Inspect → Subscribe → Deploy → Invoke

It can provide:

- Additional open models
- Specialized models
- Third-party models
- Managed endpoints
- AWS integration
- Unified access through Bedrock APIs

AWS documentation describes Bedrock Marketplace models as deployable to managed endpoints and accessible through Bedrock APIs; subscription terms and infrastructure costs need to be considered separately.

What Does “Marketplace” Actually Mean?



A marketplace is not simply: “A website containing models.”

It can combine:

Discovery

→ model catalog

Commercial transaction

→ subscription / pricing / terms

Deployment

→ managed endpoint

Infrastructure

→ compute selection

API access

→ standardized invocation

Enterprise controls

→ identity, access, governance

This is why marketplaces increasingly resemble *AI infrastructure procurement platforms*.



The catalog can contain models such as:

- DeepSeek-R1
- DeepSeek-R1-Distill variants
- Gemma variants
- Mistral variants
- Phi variants
- IBM Granite
- Falcon
- specialized models
- multilingual models
- domain-specific models

AWS's model compatibility documentation lists numerous marketplace models and indicates which support APIs such as Converse and streaming.

Enterprise implication: capability is only one filter; API compatibility and deployment characteristics matter too.

Model Garden = Google's model discovery and deployment environment.

Enterprise teams can evaluate:

- Google models
- Partner models
- Open models
- Specialized models

The attraction is not merely model choice.

It is the combination of:

Model + Cloud + Data + Security + Deployment + Operations

For an enterprise already standardized on Google Cloud, platform integration can outweigh small differences in model benchmark scores.

For Microsoft/Azure-centric enterprises, the model catalog becomes part of a larger platform strategy.

Evaluate:

- Available model families
- Azure integration
- Identity
- Networking
- Security
- Monitoring
- Application integration
- Model customization
- Deployment options

Strategic question:

Is the organization buying a model—or standardizing its AI platform around Azure?



NGC matters particularly when the enterprise operates GPU infrastructure.

Think: Model → Container → CUDA → TensorRT → GPU
Rather than: Prompt → API

NGC can matter for:

- optimized containers
- inference software
- model assets
- GPU deployment
- performance optimization
- reproducible infrastructure

Key distinction: *NGC is closer to the AI infrastructure supply chain than a simple SaaS model marketplace.*

Instead of self-hosting Llama/Qwen/Mistral:
Enterprise → Hosted inference provider → Open model

Examples:

- Together AI
- Fireworks AI
- Replicate
- Groq
- other specialized inference providers

Advantages:

- Faster deployment
- No GPU procurement
- No serving infrastructure
- Access to many open models
- Easy experimentation

Risk:

You still depend on another provider.

Category	Models
Proprietary frontier	GPT, Claude, Gemini
Open/open-weight	Llama, Mistral, Qwen
Reasoning-focused	DeepSeek and other reasoning variants
Efficient	Gemma, Phi
Enterprise-oriented	Granite
Embeddings	BGE, E5, provider-specific embedding models
Vision	Multimodal / vision-language models
Speech	Whisper and commercial speech models

GPT vs Claude vs Gemini



Dimension	GPT	Claude	Gemini
Provider	OpenAI	Anthropic	Google
Access	API / platforms	API / platforms	API / Vertex
Strong candidate	General- purpose	Reasoning / coding	Multimodal
Enterprise consideration	Ecosystem	Capability / long-context workflows	Google ecosystem
Deployment	Managed	Managed	Managed
Open weights	No	No	No

Model family	Enterprise attraction
Llama	Broad ecosystem, many deployment options
Mistral	Efficient open-model ecosystem
Qwen	Strong multilingual/coding ecosystem
DeepSeek	Reasoning/coding and open-model interest

Not every enterprise task needs a frontier model.

Examples: Classification → small model

Information extraction → small model

Document routing → small model

Routine summarization → small/mid model

Complex reasoning → premium model

A mature enterprise architecture therefore becomes:

*Small model → Medium model → Premium model
based on task complexity.*

“Llama” is not a procurement specification.

Specify:

- Model family
- Exact version
- Parameter count
- Instruct/base variant
- Quantization
- Context length
- Inference provider
- Region
- License
- Endpoint
- Pricing
- SLA
- Evaluation version

Model identity must be reproducible.

Same Model $\neq$ Same Enterprise Product



Suppose the same open model is available through:

Hugging Face weights

Provider A

Provider B

AWS managed deployment

Enterprise GPU cluster

The model may be identical.

The enterprise product is not.

Differences can appear in:

- latency
- throughput
- cost
- availability
- batching
- quantization
- data handling
- monitoring
- SLA

If an organization downloads weights, it may receive:

- Model parameters
- Configuration
- Tokenizer
- Documentation
- Inference examples

It does not automatically receive:

- training data
- enterprise SLA
- production monitoring
- security controls
- guaranteed performance
- unrestricted commercial rights
- support

Open weights transfer more responsibility to the enterprise.



Check:

- License name
- License version
- Commercial-use permissions
- Modification permissions
- Redistribution
- Derivative models
- Acceptable-use policies
- Trademark requirements
- Additional provider terms
- Third-party components

The legal object is the exact model release—not the marketing label “open.”

	Proprietary API	Open/open-weight
Weights	Provider controlled	Potentially available
Deployment	Provider	Provider or enterprise
Infrastructure	Low burden	Potentially high
Portability	Lower	Higher
Customization	Provider dependent	Greater
SLA	Provider	Must engineer/contract
Initial speed	High	Medium
Operational responsibility	Low	Higher



A model can rank highly on:

- MMLU
- GPQA
- coding benchmarks
- reasoning benchmarks

and still fail:

- company terminology
- internal policies
- Indian languages
- document grounding
- tool calling
- structured output
- security constraints

Benchmark score = screening signal

Private evaluation = procurement evidence

Include:

- 20–30% routine cases
- 20% complex cases
- 20% domain-specific cases
- 10–20% adversarial cases
- 10–20% historical failures

Also include:

- multilingual examples
- long documents
- structured output
- RAG questions
- tool-use scenarios

Keep a locked test set for final comparison.

Do not evaluate only: Model output

Evaluate:

Retrieval

Prompt

Model

Tool calls

Output validation

Human review

Business outcome

Example:

A chatbot that produces excellent answers but retrieves the wrong policy document is still a failed enterprise system.

Dimension	Example metric
Correctness	Human / rubric score
Grounding	Supported claims %
Hallucination	Critical hallucination rate
Instruction following	Task compliance
Structured output	Valid JSON / schema rate
Tool use	Successful tool-call rate
Safety	Policy violation rate
User acceptance	Human acceptance rate

Measure:

TTFT - Time to first token

P50 - Typical latency

P95 - Tail latency

P99 - Extreme tail

Throughput

Requests/tokens per second

Example:

Model A:

- $p_{50} = 1.2$ sec
- $p_{95} = 4.8$ sec

Model B:

- $p_{50} = 1.5$ sec
- $p_{95} = 2.2$ sec

For an enterprise application, B may be better, despite A having lower average latency.

Total AI cost includes:

Model tokens

Retries

GPU / endpoint

RAG infrastructure

Observability

Engineering

Human review

Incident handling

Therefore:

₹ / successful business task is often more meaningful than:

₹ / million tokens

Suppose: 1 million requests/month

But self-hosting only wins if:

- utilization remains high
- GPU capacity is efficiently used
- engineering cost is controlled
- demand is predictable

TCO is a workload calculation, not a model-label calculation.

Cost	API model	Hosted open model	Self-hosted
Model/inference	₹12L	₹8L	—
GPU	Included	Included	₹6L
Platform	₹1L	₹1.5L	₹3L
Human review	₹3L	₹3L	₹3L
Total	₹16L	₹12.5L	₹12L

API vs Managed Open Model vs Self-Hosted



	API	Managed open model	Self-hosted
Time to pilot	Very fast	Fast	Slow
Control	Low	Medium	High
GPU operations	None	Low	High
Portability	Low	Medium	High
Scaling burden	Provider	Provider/platform	Enterprise
Best for	Validation	Enterprise production	Scale/control

Self-host when:

- utilization is high
- workload is predictable
- data-control requirements are strong
- model needs customization
- GPU infrastructure already exists
- inference optimization can create savings
- engineering capability exists

Do not self-host simply because:

“Open model = free.”

The model may be free while inference is not.



Hidden costs:

- GPU idle capacity
- MLOps engineers
- patching
- model upgrades
- scaling
- monitoring
- incident response
- security
- GPU procurement
- power/cooling
- disaster recovery

A ₹0 model license can still become an expensive production platform.



Application should call:

Enterprise AI Gateway

not:

Application → Claude SDK

The gateway provides:

- authentication
- routing
- logging
- cost attribution
- policy enforcement
- model selection
- fallback
- version control

Then:

Gateway → GPT / Claude / Gemini / Llama / Qwen / Mistral

This creates technical optionality.

Example:

Simple classification

→ Phi / Gemma / small Llama

Normal RAG

→ Qwen / Mistral / Llama

Complex reasoning

→ Claude / GPT / Gemini

Highly sensitive data

→ approved private/open deployment

Latency-critical

→ low-latency inference provider

Routing converts model choice into a runtime optimization problem.

Suppose:

Model A

- Very expensive
- Excellent general reasoning

Model B

- Cheaper
- Slightly weaker reasoning

With strong retrieval:

Model B + high-quality enterprise evidence
may outperform:

Model A + weak retrieval

Therefore evaluate:

Retrieval quality + generation quality + end-to-end task success

Need	Best first choice
Change instructions	Prompting
Add current company knowledge	RAG
Add citations	RAG
Change response style	Prompting / tuning
Learn stable task patterns	Fine-tuning
Current policies	RAG
Company terminology	RAG + prompting
Stable specialized behavior	Fine-tuning

Build vs Buy: What Should the Enterprise Own?



Capability	Build	Buy
Foundation model	Rarely	Usually
Commodity inference	Rarely	Usually
Enterprise data pipeline	✓	
Evaluation framework	✓	Tools can assist
Business workflow	✓	
Domain logic	✓	
Security policy	✓	
Model gateway	Often	Platform may accelerate

Requirements:

- PII
- auditability
- current product policies
- high volume
- low latency

Possible architecture:

Customer → RAG → Small/Mid Model

Complex cases:

→ Premium model

Sensitive cases:

→ Approved private route

Evaluation metric:

Cost per successfully resolved customer interaction
not simply token price.

Evaluate:

- code completion
- repository understanding
- code generation
- test generation
- debugging
- security vulnerabilities

Metrics:

Accepted code changes

Tests passed

Developer time saved

Security findings

Cost per accepted change

Compare:

- GPT
- Claude
- Gemini
- Qwen
- DeepSeek
- Llama-based coding models

Data:

- equipment manuals
- maintenance logs
- engineering documents
- troubleshooting procedures

Architecture:

Document repository

- Access-controlled RAG
- Reranker
- Model
- Citation validation

Important:

The model should not become the source of truth.

The controlled enterprise documents remain the source of truth.

Do not assume:

“High English benchmark = high multilingual performance.”

Test:

- English
- Hindi
- regional languages
- code switching
- local names
- local terminology
- translated technical content

Also measure:

Tokenization cost

because the same sentence can have very different token counts across models and languages.

Criterion	Weight
Business quality	25%
TCO	20%
Security/data	15%
Latency/reliability	10%
Integration	10%
License/legal	10%
Portability	10%

Weighted Model Selection Example



Criterion	Weight	Model A	Model B	Model C
Quality	25%	4.6	4.3	4.5
TCO	20%	3.0	4.7	4.0
Security	15%	4.5	4.2	4.5
Latency	10%	3.5	4.8	4.0
Integration	10%	4.4	4.1	4.6
Legal	10%	4.0	4.5	4.0
Portability	10%	2.8	4.7	4.2

Where Vendor Lock-In Actually Happens



Application lock-in

Provider-specific SDKs



Prompt lock-in

Provider-specific prompting behavior



Data lock-in

Vendor-specific stores/artifacts



Platform lock-in

Vendor monitoring/orchestration



Commercial lock-in

Pricing and contracts

Mitigation

- model gateway
- portable data
- versioned prompts
- independent evaluation



Enterprise model lifecycle:

Discover

→ Evaluate

→ Approve

→ Deploy

→ Monitor

→ Re-evaluate

→ Replace

→ Retire

Track:

- model version
- provider
- license
- benchmark results
- latency
- cost
- incidents
- replacement model

AWS itself exposes model IDs, availability and deprecation information as part of its model management documentation.

Model retirement should be treated as a planned business migration.



BUY

- Foundation-model capability
- Commodity inference
- Commodity AI infrastructure
- Specialized models where buying is cheaper

BUILD

- Proprietary data pipelines
- Enterprise RAG
- Business workflows
- Evaluation systems
- Domain-specific logic
- AI governance

PROTECT

- Model portability
- Data ownership
- Evaluation datasets
- Model/version records
- Second provider
- Exit strategy

Final principle

Do not build a foundation model simply because you can.

Do not buy an AI platform simply because it is convenient.

Build the capabilities that differentiate the business—and keep the model layer replaceable.